S2 HARNESS QUALITY AND TRAJECTORY AUDITING

We audit individual harnesses and their evolution across completed optimization iterations. The framework records both deterministic static analysis and LLM-based semantic judging separately, then tracks each result longitudinally.

Intrinsic harness evaluation. Static evaluation ingests the candidate harness and, only for token-efficiency comparison, a fixed seed harness. Semantic judging compares the candidate harness using a fixed rubric. Neither component reads code samples, labels, predictions, or downstream performance. The trajectory auditor separately reads the completed-iteration history and saved metrics for reporting. Thus, intrinsic evaluation determines whether a harness is structurally and semantically suitable for vulnerability analysis, rather than its efficacy in vulnerability identification. The semantic judge is prohibited from inferring accuracy, precision, recall, or vulnerability-detection performance.

For each harness h_i, the evaluator records four distinct values:

$$(Q_{\text{static}}, Q_{\text{semantic}}, E_{\text{static}}, E_{\text{semantic}})$$

Q_{static} and Q_{semantic} are quality scores, while E_{static} and E_{semantic} are eligibility decisions. The framework does not combine them into a composite score.

Quantitative static quality. Q_{static} ranges from 0 to 100 and comprises four components:

- Template integrity: 40 points
- Schema mechanics: 30 points
- Format hygiene: 15 points
- Seed-relative token efficiency: 15 points

Template integrity checks that the harness is valid text, contains exactly one code placeholder, and has no malformed or unknown template variables. Schema mechanics validates that the vulnerability-report structure is present and complete. Format hygiene checks duplicate instructions and headings, whitespace and line-ending consistency, and Markdown-fence balance. Token efficiency compares the candidate harness with the fixed seed harness.

Qualitative semantic quality. A modular LLM-as-judge evaluates the candidate harness against nine rubric dimensions: role alignment; task identity and primary task; input-object and untrusted-code boundary; SCoT Sequence, Branch, and Loop scaffold; scope and grounding; prohibited behavior; output-contract semantics; internal consistency and instruction precedence; and organization and maintainability. Each dimension is rated from 0–4. Ratings from valid judge attempts are averaged by dimension and weighted to produce Q_{semantic} on a 0–100 scale.

Eligibility criteria. Eligibility acts as a minimum validity gate distinct from quality scoring. E_{static} requires the essential template and schema conditions. E_{semantic} requires no critical semantic failure and average ratings of at least 2 out of 4 for the primary task, SCoT scaffold, scope/grounding, and output-contract dimensions. A harness can therefore receive a relatively high quality score yet remain ineligible.

Harness trajectory auditing. For every completed optimization iteration, the audit records static and semantic quality, eligibility, duplicates identified from normalized harness text, changed-line counts, optimizer change summaries, and saved downstream metrics. The latter include pairwise accuracy, accuracy, precision, recall, F1, parse failures, and sample count when available. The audit does not impose monotonic improvement or construct a combined trajectory score; each measurement remains separate so that changes across iterations can be examined directly.

Table S2.1 reports every validation Pareto-frontier harness (pairwise accuracy vs. joint hit rate). Ratings are the GPT-OSS-120B judge's 0–4 scores averaged over its valid runs; FP and Flagged compare the seed with each harness on the 404 validation functions.

Table S2.1: Every Pareto-frontier harness: validation metrics, judge ratings, and length.

Agent + Model	Iter.	Pairwise	Joint	SCoT	Prohib.	Output	Judge	Length	FP	Flagged
Codex + Inkling-Small	6	20.8	19.3	4	4	4	99.4	3.8×	185 → 123	378 → 262
	8	23.3	12.2	4	4	4	98.8	3.7×	185 → 117	378 → 256
	12	27.7	9.4	4	4	4	100.0	4.8×	185 → 104	378 → 243
Codex + DeepSeek V4 Flash	1	24.3	28.2	4	4	4	100.0	4.1×	160 → 70	341 → 178
	5	28.7	24.2	4	4	3	90.4	12.0×	160 → 89	341 → 224
	9	30.2	22.1	4	4	4	91.2	19.3×	160 → 64	341 → 175
	11	36.1	20.9	4	4	4	100.0	13.9×	160 → 65	341 → 191
	14	33.7	21.5	4	4	4	99.4	21.5×	160 → 72	341 → 191
Claude Code + Opus 4.6	2	31.2	31.3	4	4	4	100.0	7.8×	170 → 85	357 → 225
	3	33.7	28.6	4	4	4	100.0	7.7×	170 → 85	357 → 232
Claude Code + GLM-5.3	6	31.2	34.1	4	4	3.5	98.0	16.2×	134 → 69	297 → 192

Table S2.2 lists every harness that the judge rated below the top score on the prohibited-content or output-format dimension.

Table S2.2: Harnesses below the top rubric rating (GPT-OSS-120B judge, two runs per harness; a rating averages the valid runs). ‡ marks harnesses on the validation Pareto frontier; the only prohibited-content case is not on the frontier, and the two frontier cases concern output format only.

Harness	Dimension	Rating	Finding
Inkling-Small, iter 14	Prohibited content	3.0	Prefers "not vulnerable" when source or sink is unquoted
Inkling-Small, iter 13	Output format	3.5	SCoT reasoning required, then "only" a JSON object
DeepSeek V4 Flash, iter 5‡	Output format	3.0	SCoT reasoning required, then "only" a JSON object
GLM-5.3, iter 2	Output format	3.5	The example output schema leaves line numbers as `null`, but the instructions require integer line numbers
GLM-5.3, iter 3	Output format	3.0	The example output schema leaves line numbers as `null`, but the instructions require integer line numbers
GLM-5.3, iter 6‡	Output format	3.5	The example output schema leaves line numbers as `null`, but the instructions require integer line numbers